

Frequent Itemset Analysis in Yelp Reviewers

Michael Davis
CSC-467
West Chester University
West Chester, United States
mdavis1054470@wcupa.edu

Abstract— The misuse of online review systems, such as Yelp, introduce a challenge for maintaining trust in the user and fairness when assessing a business. One malicious behavior of note known as review-bombing involves a coordinated effort to leave low score reviews which can be potentially damaging to a business’s reputation. This analysis investigates the extent and nature of these occurrences by leveraging the use of frequent itemset analysis by method of the A-Priori algorithm principles to identify co-occurring review patterns among suspected users. Use of Yelp’s publicly available datasets, the analysis utilizes user relationships through the platform’s friend system to uncover potential coordinated behavior. To support scalability due to the sheer size of these available datasets, use of Apache Spark’s capabilities in processing large-scale sets of data are in full force. Using Spark’s Resilient Distributed Datasets (RDDs) for efficient data handling. The findings revealed 331 frequent user pairs with a confidence interval of 75% or higher, suggesting instances of likely correlated review patterns. This methodology provides a framework for detection of bad actors on the platform, contributing to the integrity of both online review platforms such as Yelp and supporting businesses. (*Abstract*)

Keywords—frequent itemset, review-bombing, Latent Dirichlet Allocation (LDA), Resilient Distributed Dataset (RDD), Javascript Object Notation (JSON) (*key words*)

I. INTRODUCTION

This analysis looks into potential review-bombing occurrences through the use of frequent itemset analysis techniques to look at the publicly available Yelp Dataset to highlight suspect users engaging in the activity and other user accounts involved. Yelp is a review platform where users can make an account and review a business leaving their thoughts and a star rating. In the scope of this report, I explore patterns of frequent co-occurrence in low-rated reviews left by both users and their friends. By combining data manipulation techniques and frequent itemset analysis, this work aims to achieve as follows:

1. Highlight review patterns that suggest coordinated activity.
2. Identify frequent pairs and groups of users involved in leaving low score reviews on businesses.
3. Establish a confidence interval to measure the rate of occurrence of users reviewing the same business.

Assessing known user relationships is made possible through the use of Yelp’s friend functionality as each user in the dataset has a friend list with identifiable identification strings allowing us to accurately reference any review left by a user or their friends.

II. MOTIVATION

Validating the integrity of online review systems such as those seen on Yelp is paramount for businesses, especially small businesses that often rely on positive customer feedback and word of mouth to garner new customers. Negative reviews can be severely damaging to a small business in deterring prospective future customers and causing long-term damage. When bad actors exploit these systems—whether through the use of alternate accounts, coordinated attacks amongst friends, or other means—they undermine the credibility of these platforms and cause undue harm on the reputation of the business in question.

An account is only allowed one review on a business, this is a fundamental aspect of the online platform. That is the scope a user is allowed feedback. An attempt to undermine that limitation through coordinated efforts such as review-bombing campaigns are representative of illegitimate bad faith behavior that not only causes harm to the business in question, but also damages the trust in the platform itself.

The methodology outlined in this report provides an approach to detecting these types of activities and enabling platforms like Yelp to be able to take appropriate action as necessary. In protecting the integrity of the review system, this work aims to protect the livelihood of small businesses by ensuring fair and unbiased representation on the platform.

III. RELATED WORK

The Yelp Dataset Challenge inspires a multitude of impressive data mining techniques to analyze user, business and review data. A notable work was that of Jack Linshi, one of the winners of the third round of Yelp’s data challenges, whose analysis focused on uncovering underlying topics in review text by utilizing their modified approach of Latent Dirichlet Allocation (LDA) [1]. LDA is a probabilistic model typically associated with artificial intelligence used to assess language and uncover underlying topics within the text of a document. Linshi’s approach utilizes this technique to draw parallels between positive and negative words in review text to create things such as likely review scores based on word associations.

The research looks to provide insight much like how a frequent itemset analysis does in how it analyzes co-occurrence to find underlying trends. Both methods rely on high-probability associations to identify a meaningful pattern. For example, Linshi’s research identified words like “walk, pizza, seat, pie, cheese” as words that frequently co-occur in reviews about flavor. [1]

Similarly, frequent itemset analysis can reveal associations between two items, however, instead of words, the focus is on user review behavior and relationships. Just like probability-based word pairs in LDA suggest underlying topics, high-confidence user pairings in frequent itemset analysis can suggest coordinated behaviors or relationships. By applying this approach to user reviews, it becomes possible to identify underlying connections such as possible collusion between users or systematic review bombing against a business.

IV. DESCRIPTIVE ANALYSIS

A. Dataset Overview

The analysis begins with three datasets in our focus provided by Yelp: user, review and business datasets. Each dataset contains attributes critical to understanding relationships between users and potential coordinated behaviors as well. Notable attributes include:

- User Dataset
 - User ID: A unique identifiable alphanumeric string assigned to every user.
 - Name: The user’s self-assigned username.
 - Friends: A list of associated users listed by their own unique user ID representing declared friendships.
- Review Dataset
 - Review ID: A unique identifiable alphanumeric string assigned to every review.
 - User ID: The author of the review.
 - Business ID: The business being reviewed.
 - Star rating given by the reviewing user.
 - Review text.
- Business Dataset
 - Business ID: A unique identifiable alphanumeric string assigned to every business.
 - Name: The name of the business.
 - Number of reviews received.
 - Overall star rating.

The data available to use contains a very cumbersome, I look at three different available datasets: user, review and business. Each of these datasets contain important attributes we need to utilize. A user, review and business each have an ID associated with them which consists of a randomized alphanumeric string as well as other useful identifiers such as a name users give themselves amongst some others which will be discussed shortly. First we need to pull the base datasets which we do by loading these into a Resilient Distributed Dataset or RDD. This is a manipulatable form for our data parallel to the data we are looking at. After having our data loaded, the fields

Flagged User			
Name	Business ID	Review ID	Friend Reviews (User ID, Review ID)
Amanda	BJ's Restaurant & Brewhouse	Udlabx_W_sl UOjYNAplDg	Taylor, wFwWCuRRfpwd_it we5JNTA

Fig. 1. Example data extracted from a suspected user.

necessary for inspection are extracted from our RDD. While not all of these are used for our analysis such as a user’s name is not used in a capacity that affects our outcome, it does aid in giving us a readable presentation of the data.

B. Narrowing the Scope

From these, we need to narrow our scope further in order to maintain a manageable amount of data as our set is massive and still bringing a lot of information even with all other attributes not mentioned not being brought over. The reviews are limited to only include one to two star reviews, which we will later only use the one star reviews. At this point, a review is displayed in the format of review ID, user ID, business ID, stars given and review text. Reviews on a given business by a user and at least one friend give a better understanding of potential user relationships. We can see an instance of this in Fig. 1.

This is a simplified form of what we are working with where the user and business IDs were replaced with the names of the respective user and business, we can see based this that Amanda and Taylor are both friends, have left a review with an ID we can track on the same business and these two reviews were filtered so that we know they are both one star reviews. This flagging occurrence is taken as a counted occurrence and we can sort our users by the count of hits on suspected reviews.

C. Aggregation and Initial Insights

It is naïve to assume that because a user has a lot of overlap with their friends means anything, when we look at the ratio of this relationship with the overall count of their friends’ reviews combined and the individual user’s review count to compare we find that our top user’s flagged review rate goes from being an interestingly high number of 1,315 reviews to just being approximately 0.9% of reviews in comparison to the aggregated count of all friends’ reviews. What needs to be inspected is a pattern behavior where two users co-occur and a high frequency.

V. TECHNICAL ANALYSIS

A. Preparing the Data

Careful consideration is needed when handling a dataset as large as the Yelp dataset, as scalability quickly becomes a concern due to the overbearing load on data operations. The analysis performed heavily utilizes Spark to carry out these heavy operations. The data is formatted in JavaScript object notation (JSON) which is loaded into an RDD where it can be further processed by use of mapping which is a method of transforming an existing RDD into a new one, in this case we use this functionality to extract fields of interest from each of the Yelp datasets. Attributes mentioned earlier that belong to users, businesses and reviews such as their corresponding ID. This information is placed into a tuple which is akin to placing objects into a bucket, where we format the attributes we are using in a convenient format for later.

User ID	Name	Flagged Review Count	Aggregated Friend Reviews	Individual Review Total
_BckWyKQL16ndp8dggh2kNA	Karen	1,315	146,544	4,274
ET8n-r7glWYqZhuR6GcdNw	Michelle	1,095	381,612	2,256
bj5FtCtZX3Zzac2_2PJJA	Bill	475	101,598	1,512
1HM81n6n4iPIFU5d2Lokhw	Shannon	461	32,947	2,317
CfX4sTIFFNaRchNswqhVfg	Christopher	399	37,223	1,405
E4BsVQnG5zetbwv2x8QIWg	Charles	395	34,817	726
pou3BbKslozfH50rxmnMew	Brett	367	204,627	2,812
Ase_kJIYuT6yOsqqVPuWUA	Teneha	319	71,234	1,134
bYENop4BuQepBjM1-BI3fA	Steven	317	82,785	1,562
D8CF2H3DYtNJlZR0IITMHQ	ThaDragonSourceRa®	305	19,989	323

Fig. 2. Sample list of top flagged users when sorted by flagged review count

A filter operation is conducted on our extracted review fields in which we target the fourth element, the number of stars given as a rating in said review. This is checking for reviews that gave either a score of one or two stars and discarding any results that give a higher score. Our given low rated review RDD already has the given user's ID but re-keying the user ID helps for our comparison operations so we can more easily join it with our extracted user fields. This combines a given review with the extracted user information so we can build a database of sorts to profile users and their low reviews.

B. User and Friend Pairing

Building the profile we require is still missing an essential aspect which is each user's friends' review behavior respectively so we can build our foundation for further analysis. We split the list of friend IDs from the user fields and pairing that with our low score reviewer users. There is a challenge in both pairing a user and their friends' reviews as well as maintaining these pairs while also pairing each of these reviews to the business they are tied to so we make further key pairs based on business as well with our low score reviews. Identification of reviews from a user's friends for the same business is done by getting the list of friends of a user and all reviews for a given business and filtering them to only include reviews written by the user and their friends. This starts building our working format. We can further narrow our working set by only looking at instances where at least one friend has reviewed the same business as the user. We further polish our dataset by converting it to a structured dictionary which is grouped by the user ID. This dictionary contains the user's review, business ID, and friend user IDs and reviews respectively. The friend user ID and review ID is effectively a nested tuple that represents a user ID and review ID which has been associated with the key user ID.

C. Checkpointing and Validation

A challenge is hit in managing the massive amount of data still in use, which results occasionally in a hanging operation and starting from the beginning. To eliminate this a checkpoint is made at this point. The results are saved so they can be pulled again where the analysis is left off and serialized to JSON strings for improved readability as the nested tuples begin creating a large amount of text. This checkpoint is done at multiple points along the process to avoid difficulties in the analysis.

Testing is performed by isolating one entry in our resulting users that are still in our dataset, the user's friends reviews are checked by ID through a filter to give only reviews associated with the user and their friends. Manual review confirms no malformed structure in the format and all passed data is within the scope the filter operations aimed for. Since the isolated testing has worked scaling is done by using the full RDD of user-friend paired reviews and a count is built of all nested friend ID, friend review tuples for a descending list of hits on a given user. Names are pulled for the users in this set and displayed alongside the user's ID.

D. Aggregating Counts for Users and Friends

Each user's list of friends is iterated through by creating an RDD that maps the user to their set of friends, reviews not included, and the aggregated count of each friends' reviews are added together to get a total count of any user's total friend reviews. A flat map operation is performed which is effectively a reversal of the key pair of users and friend to make it friend and user. Reviews are linked to these friend IDs. This reverse order is then mapped with the value one to count. Now a given user's friends' review counts are easily aggregated to give the total reviews across all friends to compare to the user's overall count and the flagged user friend pairs from low score reviews on the same business. These counts are all performed by way of reduce by key operations. The results are sorted by flagged review counts and the comparison shows that making correlations based on these ratios will need further refinement as shown in Fig. 2.

E. Frequent Itemset Analysis

At this stage it is clear a frequent itemset analysis is required to properly gauge the relationship between users and their friends. Baskets are created where each basket can be pictured as a group of associated suspect users. A support threshold is set at eight and each bucket is paired with users for counting. A frequent pair is defined as an RDD of a given user and occurrence count and a list is collected of these. For these baskets, it is ensured that the secondary value is greater than the key value to avoid duplicates. This gives instances of two different users co-occurring by count. Pairs are counted exactly as the items building said pairs were and a list is made of those for the generation of triplet pairs, which is three different users co-occurring.

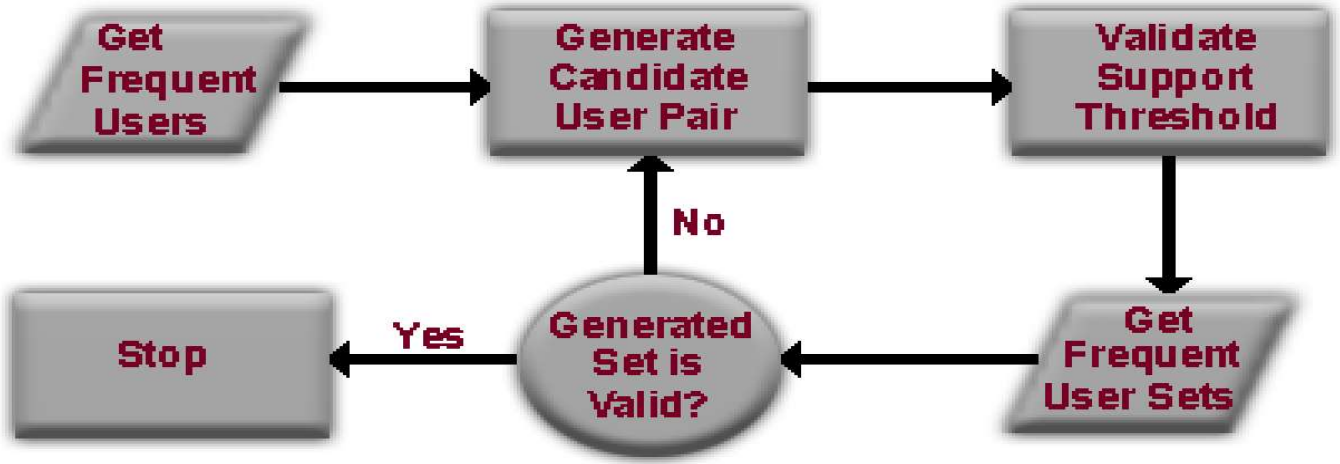


Fig. 3. A-Priori Algorithm Logic Gate

The process follows the A-Priori algorithm by doing this, the support threshold is the method of pruning both in generating the frequent pairs and triplet sets. Triplets which are iteratively generated using only those frequent subsets. A-Priori algorithm follows the format seen as approximate as possible in the Spark environment as seen in Fig. 3.

Confidence is alike probability in the context we are assessing. A confidence of 100% states that the pair occurs 100% of the time. That is to say, every time the key user leaves a review, the friend in their pair will always have also left a review. In our case these are still one star reviews. This is calculated by flattening our pairs into keyed tuples and joining the pair counts with all individual user counts.

$$C = \frac{A \cup B}{A}$$

This calculation allows us to display a confidence interval of a pair of users co-occurring as previously described, which gives a sizeable list. The numerator is the aggregated count of both items of the set divided by the denominator, the count of the key value. There are 1,446 frequent pairs generated out of the users in the Yelp dataset at the given support threshold. Out of these, 30 pairs had the highest confidence interval of 100%. Telling us that they always review the same business with a low score. 331 had a confidence interval of 75% or higher. Triplets were also assessed and found 11 users with a confidence of 100% and 107 with 75% or higher. All generated pairs had at least 12 occurrences which suggests that each user in the key pair left at least 12 reviews on various businesses.

VI. CONCLUSION

The analysis performed indicates that while review bombing may not be rampant, the existence of such may still be highly damaging for the businesses that are affected. The

identification of 331 frequent pairs with a confidence interval of 75% or higher highlights user relationships that may warrant further investigation. Among these, a subset demonstrates highly coordinated behavior, indicating potential bad faith actions or alternate user accounts.

The findings in this analysis provide a few potential applications which networks such as Yelp can leverage as follows:

1. Detecting and mitigating coordinated review bombing to protect businesses from unfairly damaging reviews.
2. Understanding user relationships to uncover patterns that may indicate unknown connections between users beyond the built-in friend capability on the platform.
3. Maintaining a higher platform integrity to expand trust in the review system for users and businesses.

While the analysis performed focuses primarily on one use case, this methodology can be easily adapted and extended to other scenarios. By utilizing techniques such as those performed in this report, platforms like Yelp can further reinforce the integrity and accuracy of online reviews to support not only the health of their platform, but the health of small businesses as well.

REFERENCES

- [1] J. Linshi, "Personalizing Yelp Star Ratings: a Semantic Topic Modeling Approach." Yale University, 2014. Yelp. [Online]. Available: https://www.yelp.com/html/pdf/YelpDatasetChallengeWinner_PersonalizingRatings.pdf